

Algorithmen und Datenstrukturen

Übung 7: Sortieren

Robert

2026-06-16

Wiederholung

Sortieralgorithmen

BubbleSort

InsertionSort

SelectionSort

BubbleSort

- BubbleSort vergleicht zwei benachbarte Zahlen
- wenn die linke Zahl größer ist, werden die zwei Zahlen getauscht
 - dadurch wandern die größten Zahlen ans Ende des Arrays

```
1 public void bubblesort(int[] a){
2     int n = a.length;
3     for(int i = 0; i < n; i++) {
4         for(int j = 0; j < n-i-1; j++){
5             if(a[j] > a[j+1]) swap(a,j,j+1);
6         }
7     }
8 }
```

- BubbleSort ist ein adaptiver Sortieralgorithmus
 - das heißt, die Laufzeit oder Anzahl der Vergleiche hängt davon ab, wie vorsortiert das Array bereits ist
 - wenn das Array schon teilweise vorsortiert ist, schneidet BubbleSort besser ab
- BubbleSort ist außerdem stabil (die Reihenfolge gleicher Elemente wird beibehalten)

Komplexitätsanalyse

- Vertauschungen:
 - Best Case: 0 Swaps (bereits sortiert)
 - Worst Case: $\frac{1}{2}(n^2 - n)$ Swaps
 - Durchschnitt: $\frac{1}{4}(n^2 - n)$ Swaps
 - eine Vertauschung in einem Array ist immer konstant
 - Komplexität bei Vertauschungen: $O(1)$

- Vergleiche:
 - die Anzahl der Vergleiche ist unabhängig von der Vorsortierung
 - Worst Case, Average Case und Best Case sind identisch
 - Anzahl: $\sum_{i=1}^n (n - i) = \sum_{i=0}^{n-1} i$
 - Komplexität bei Vergleichen: $O(n^2)$

InsertionSort

- InsertionSort sortiert das Array von vorne nach hinten, indem die kleinen Elemente vorne an die richtige Stelle eingefügt werden
 - funktioniert ähnlich wie BubbleSort, nur wandern die kleinen Elementen an den Anfang
- der unsortierte Rest wandert nach hinten

```
1 public void insertionsort(int[] a){
2     for(int i = 1; i < a.length; i++) {
3         int key = a[i];
4         int j = i;
5         while(j > 0 && a[j-1] > key){
6             a[j] = a[j-1];
7             j--;
8         }
9         a[j] = key;
10    }
11 }
```

- InsertionSort ist auch ein adaptiver Sortieralgorithmus
- ist auch stabil

Komplexitätsanalyse

- Vertauschungen:
 - Best Case: 0 Swaps (bereits sortiert)
 - Worst Case: $\frac{n(n-1)}{2} \in O(n^2)$
 - Average Case: $\frac{n(n-1)}{4} \in O(n^2)$
- Vergleiche:
 - abhängig von der Vorsortierung
 - Average Case und Worst Case: $O(n^2)$
 - Best Case: $O(n)$ (bereits sortiert)

SelectionSort

- SelectionSort sucht sich immer das kleinste Element und verschiebt es nach vorne ins Array
- so entsteht vorne ein sortierter Teil und hinten ein unsortierter Teil, in dem immer das kleinste Element raus gesucht und vorne eingefügt wird

```
1 public void selectionsort(int[] a){
2     for(int i = 0; i < a.length-1; i++) {
3         int min = i;
4         for(int j = i+1; j < a.length; j++){
5             if(a[j] < a[min]) min = j;
6         }
7         if(min>i) swap(a,i,min);
8     }
9 }
```

- SelectionSort ist weder adaptiv noch stabil

Komplexitätsanalyse

- Vertauschungen:
 - Best Case: 0 Swaps (bereits sortiert)
 - Worst Case: $n - 1$ Swaps
 - Average Case: $n - 1$ Swaps
- Vergleiche:
 - es gibt n Durchläufe und in jedem Durchlauf finden $n - i$ Vergleiche statt
 - insgesamt: $\frac{n(n-1)}{2} \in O(n^2)$

Aufgabe 1

1a)

Sortiere die sechs Gebäude mit den meisten Etagen mit BubbleSort. Zähle die Vergleichs- und Vertauschoperationen

Rang	Gebäude	Stadt	Höhe	Etagen	Baujahr
1	Burj Khalifa	Dubai (VAE)	828 m	163	2010
2	Shanghai Tower	Shanghai (VR China)	632 m	128	2015
3	Makkah Royal Clock Tower	Mekka (Saudi-Arabien)	601 m	120	2012
4	Ping An Finance Center	Shenzhen (VR China)	599 m	115	2016
5	Lotte World Tower	Seoul (Südkorea)	555 m	123	2017
6	One World Trade Center	New York City (USA)	541 m	94	2014
7	Guangzhou CTF Finance Center	Guangzhou (VR China)	530 m	111	2016
8	Tianjin CTF Finance Center	Tianjin (VR China)	530 m	97	2020
9	China Zun Tower	Peking (VR China)	528 m	108	2018
10	Taipei 101	Taipei (Taiwan)	508 m	101	2004

→ [163, 128, 120, 115, 123, 94]

- [128, 163, 120, 115, 123, 94]
- [128, 120, 163, 115, 123, 94]
- [128, 120, 115, 163, 123, 94]
- [128, 120, 115, 123, 163, 94]
- [128, 120, 115, 123, 94, 163]
- [120, 128, 115, 123, 94, 163]
- [120, 115, 128, 123, 94, 163]
- [120, 115, 123, 128, 94, 163]
- [120, 115, 123, 94, 128, 163]
- [115, 120, 123, 94, 128, 163]
- [115, 120, 94, 123, 128, 163]
- [115, 94, 120, 123, 128, 163]
- [94, 115, 120, 123, 128, 163]

- $\rightarrow$ 13 Vertauschungen
- $\sum_{i=0}^{6-1} i = 15$ Vergleiche

1 b)

Sortiere die sechs Gebäude mit den meisten Etagen mit SelectionSort. Zähle die Vergleichs- und Vertauschoperationen.

[163, 128, 120, 115, 123, 94]

Vertauschoperationen

- [94, 128, 120, 115, 123, 163]
- [94, 115, 120, 128, 123, 163]
- [94, 115, 120, 123, 128, 163]
- → 3 Vertauschungen

Vergleiche

- $\frac{6 \cdot (6-1)}{2} = 15$ Vergleiche

[163, 128, 120, 115, 123, 94]	Vergleich von:
[163 , 128 , 120, 115, 123, 94]	128 163
[163, 128 , 120 , 115, 123, 94]	120 128
[163, 128, 120 , 115 , 123, 94]	115 120
[163, 128, 120, 115 , 123 , 94]	123 115
[163, 128, 120, 115 , 123, 94]	94 115
[94 , 128 , 120 , 115, 123, 163]	120 128
[94 , 128, 120 , 115 , 123, 163]	115 120
[94 , 128, 120, 115 , 123 , 163]	123 115
[94 , 128, 120, 115 , 123, 163]	163 115
[94 , 115 , 120 , 128 , 123, 163]	128 120
[94 , 115 , 120 , 128, 123 , 163]	123 120
[94 , 115 , 120 , 128, 123, 163]	163 120
[94 , 115 , 120 , 128 , 123 , 163]	123 128
[94 , 115 , 120 , 128, 123 , 163]	163 123
[94 , 115 , 120 , 123 , 128 , 163]	163 128

1 c)

Sortiere die sechs Gebäude mit den meisten Etagen mit InsertionSort. Zähle die Vergleichs- und Vertauschoperationen.

[163, 128, 120, 115, 123, 94]

Vertauschoperationen

- [128, 163, 120, 115, 123, 94]
- [120, 128, 163, 115, 123, 94]
- [115, 120, 128, 163, 123, 94]
- [115, 120, 123, 128, 163, 94]
- [94, 115, 120, 123, 128, 163]
- → 5 Vertauschungen

Vergleiche

- 14 Vergleiche

[163, 128, 120, 115, 123, 94]	Vergleich von:	Änderung
[163, 163 , 120, 115, 123, 94]	163 128	1
[128 , 163, 120, 115, 123, 94]	-	2
[128, 163, 163 , 115, 123, 94]	163 120	3
[128, 128 , 163, 115, 123, 94]	128 120	4
[120 , 128, 163, 115, 123, 94]	-	5
[120, 128, 163, 163 , 123, 94]	163 115	6
[120, 128, 128 , 163, 123, 94]	128 115	7
[120, 120 , 128, 163, 123, 94]	120 115	8
[115 , 120, 128, 163, 123, 94]	-	9
[115, 120, 128, 163, 163 , 94]	163 123	10
[115, 120, 128, 128 , 163, 94]	128 123	11
[115, 120, 123 , 128, 163, 94]	123 120	12
[115, 120, 123, 128, 163, 163]	163 94	13
[115, 120, 123, 128, 128 , 163]	128 94	14
[115, 120, 123, 123 , 128, 163]	123 94	15
[115, 120, 120 , 123, 128, 163]	120 94	16
[115, 115 , 120, 123, 128, 163]	115 94	17
[94 , 115, 120, 123, 128, 163]	-	18

Aufgabe 2

2 Worst Case

Gebe für alle drei Algorithmen ein Array an, das jeweils das Worst Case Szenario darstellt.

BubbleSort

Da bei BubbleSort immer das größte Element an das Ende des Arrays verschoben wird, ist das Worst Case Szenario ein Array, das in umgekehrter Reihenfolge sortiert ist:

[100, 90, 80, 70, 60, 50, 40, 30, 20, 10]

InsertionSort

InsertionSort verschiebt die kleinsten Elemente nach vorne. Daher ist der Worst Case hier auch ein Array mit umgekehrter Reihenfolge:

[100, 90, 80, 70, 60, 50, 40, 30, 20, 10]

SelectionSort

Da SelectionSort kein adaptiver Sortieralgorithmus ist, spielt die Vorsortierung keine Rolle. Die Anzahl der Vergleiche bleibt immer gleich, nur die Anzahl der Vertauschungen kann sich unterscheiden.

Worst Case-Beispiel:

[100, 10, 20, 30, 40, 50, 60, 70, 80, 90]

Aufgabe 3

3 a)

Folgendes Array ist gegeben, das mitten aus dem Sortiertprozess genommen wurde.

Welcher Algorithmus wurde angewendet?

[4, 5, 7, 8, 9, 11, 20, 21, 22, 23, 25, 28, 2, 1, 13, 12, 14, 15, 6, 17, 10, 29, 16, 18, 24, 19, 0, 3, 27, 26]

- das Array ist vorne sortiert
 - das heißt die kleinen Werte wandern nach vorne

⇒ InsertionSort

3 b)

Welcher Algorithmus wurde angewendet?

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 26, 24, 22, 18, 28, 27, 17, 29, 23, 21, 19, 20, 16, 25]

- das Array ist auch vorne sortiert, aber mit den kleinsten Werten
 - das heißt die kleinsten Werte werden im Array rausgesucht und vorne eingefügt

⇒ SelectionSort

3 c)

Welcher Algorithmus wurde angewendet?

[2, 5, 3, 8, 9, 14, 7, 4, 13, 15, 17, 19, 0, 10, 11, 20, 21, 1, 22, 18, 23, 16, 6, 12, 24, 25, 26, 27, 28, 29]

- das Array ist hinten sortiert
 - das heißt die größten Werte wandern nach hinten

⇒ BubbleSort